

SIMATS ENGINEERING

COURSE: COMPUTER NETWORKS (CSA0711)

CSA0711— ASSESSMENT TOOL 1 —

Assessment Tool 1 – Experiments | CO3 | 10 Marks

Course Outcome: CO3 – Analyze the performance of core network protocols such as TCP, UDP, HTTP and DNS under varying conditions.

This print-ready document contains Questions 1–4 from the supplied assessment PDF. Sample captured values are retained as sample values; replace them with your own Wireshark observations if required by the faculty.

Question

Capture packets during a TCP connection setup. Identify the three packets involved in the handshake (SYN, SYN-ACK, ACK). Demonstrate the role of each flag and how they establish a reliable connection.

1.1 Objective

To use Wireshark to capture live traffic during the establishment of a TCP connection, identify the SYN, SYN-ACK and ACK segments that make up the three-way handshake, and explain how the flags and sequence/acknowledgment numbers cooperate to establish a reliable, connection-oriented session between a client and a server.

1.2 Tools / Software Required

- Wireshark (packet capture and analysis)
- A host machine with an active Wi-Fi/Ethernet network interface
- A web browser or terminal utility such as curl/telnet to generate a fresh TCP connection

1.3 Procedure

1. Open Wireshark and select the active network interface.
2. Start a capture and apply the display filter: `tcp.flags.syn == 1 || (ip.addr == <server_ip> && tcp)`.
3. Open a web page or use curl to initiate a fresh TCP connection.
4. Stop the capture and locate the first three packets exchanged between the client and server.
5. Expand the Transmission Control Protocol section and inspect Flags, Sequence Number and Acknowledgment Number.
6. Right-click the first SYN packet and choose Follow → TCP Stream to isolate the connection.

1.4 Sample Captured Output

No.	Time	Source	Destination	Flags	Seq / Ack	Info
1	0.000000	192.168.1.10	93.184.216.34	SYN	Seq=0	49514 → 80 [SYN], Win=64240, MSS=1460
2	0.028417	93.184.216.34	192.168.1.10	SYN, ACK	Seq=0, Ack=1	Server acknowledges client SYN and sends its SYN
3	0.028562	192.168.1.10	93.184.216.34	ACK	Seq=1, Ack=1	49514 → 80 [ACK], Win=64240

Sample Captured Output:

Wireshark screenshot: Capture showing the TCP SYN, SYN-ACK and ACK packets.

1.5 Role of Each Flag / Field

Packet	Flag(s) Set	Purpose
Packet 1 (Client → Server)	SYN = 1	Client proposes to open a connection and announces its Initial Sequence Number (ISN).
Packet 2 (Server → Client)	SYN = 1, ACK = 1	Server acknowledges the client's SYN (Ack = client ISN + 1) and sends its own SYN.
Packet 3 (Client → Server)	ACK = 1	Client acknowledges the server's SYN (Ack = server ISN + 1); the connection becomes ESTABLISHED.

1.6 How the Handshake Establishes Reliability

- Sequence and acknowledgment numbers establish the starting point for byte-level ordering and help TCP detect loss, duplication and re-ordering.
- SYN/SYN-ACK options can negotiate parameters such as MSS, Window Scale and SACK.
- The exchange confirms that packets can travel in both directions before application data is sent.
- After the third packet, both endpoints enter the ESTABLISHED state and begin tracking the connection for reliable, in-order delivery.

1.7 Analysis / Discussion

The three-way handshake is SYN → SYN-ACK → ACK. The client first requests a connection, the server acknowledges the request and sends its own synchronization information, and the client confirms the server's response. The sample capture shows a round-trip interval of approximately 0.028 seconds between the first and third packets.

1.8 Conclusion

The capture confirms that TCP uses a three-segment SYN/SYN-ACK/ACK exchange to synchronize sequence numbers, negotiate connection parameters and verify bidirectional reachability before data exchange. This forms the basis of TCP's reliable, connection-oriented service.

Question 2

Capture a DNS query using UDP in Wireshark. Compare the UDP header size with TCP and note the absence of sequence numbers or acknowledgments. Discuss how this lightweight design impacts speed and reliability.

2.1 Objective

To capture a DNS lookup carried over UDP, examine the UDP header fields in Wireshark, contrast the header structure and size with a TCP header, and explain the speed/reliability trade-off resulting from UDP's minimal, stateless design.

2.2 Tools / Software Required

- Wireshark
- nslookup or dig

2.3 Procedure

7. Start a Wireshark capture and apply the filter `udp.port == 53` or simply `dns`.
8. Flush the local DNS cache so that the query is not answered from cache.
9. Run `nslookup www.example.com` or `dig www.example.com` from a terminal.
10. Select the outgoing DNS query and expand User Datagram Protocol to inspect its header.
11. Select a TCP segment from the capture and expand Transmission Control Protocol for comparison.

2.4 Sample Captured Output

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.10	8.8.8.8	DNS	70	Standard query 0x1a2b A www.example.com
2	0.021340	8.8.8.8	192.168.1.10	DNS	86	Standard query response 0x1a2b A www.example.com A 93.184.216.34

Sample Captured Output:

Wireshark screenshot: Capture showing the DNS UDP query and response.

2.5 UDP Header vs TCP Header

Field	UDP Header	TCP Header
Fixed header size	8 bytes	20 bytes minimum (without options)
Source / Destination Port	Present	Present
Sequence Number	Absent	Present
Acknowledgment Number	Absent	Present
Flags	Absent	Present (SYN/ACK/FIN/RST etc.)
Window Size	Absent	Present
Checksum	Present	Present
Connection setup	No — connectionless	Yes — 3-way handshake

2.6 Observations

- UDP has only an 8-byte fixed header, compared with TCP's 20-byte minimum header.
- UDP has no Sequence Number or Acknowledgment Number, so the transport layer does not provide built-in loss, duplication or ordering recovery.
- DNS uses its own 16-bit Transaction ID to match a response with a query and can use application-level timeout/retry.
- No TCP-style handshake is required before the DNS query is sent.

2.7 Impact of the Lightweight Design on Speed and Reliability

UDP is faster because it avoids connection setup and per-packet acknowledgments. In the sample capture, the DNS request-to-response gap is about 21 ms. The trade-off is that UDP does not guarantee delivery, ordering or duplicate protection at the transport layer. The DNS resolver handles lost queries using timeout and retry mechanisms. DNS can fall back to TCP when a response is too large for the UDP/EDNS0 payload or when reliable segmented delivery is required.

2.8 Conclusion

The 8-byte UDP header confirms that UDP minimizes overhead and latency by omitting sequencing, acknowledgment and flow-control fields. This makes UDP suitable for small, latency-sensitive DNS exchanges, while reliability responsibilities are handled at the application level.

Question 3

Simulate packet loss (e.g., disconnect briefly) and capture traffic in Wireshark. Observe how TCP retransmits lost packets while UDP does not. Explain why this difference makes TCP reliable but slower compared to UDP.

3.1 Objective

To induce packet loss during active TCP and UDP transfers, capture both in Wireshark, and compare the behavior of the two protocols, specifically TCP retransmission and UDP's lack of transport-layer recovery.

3.2 Tools / Software Required

- Wireshark
- A sustained TCP transfer such as a large file download or video stream
- A UDP stream such as `iperf3 -u` or a real-time media stream
- A way to induce brief packet loss, such as toggling Wi-Fi off/on for 1–2 seconds

3.3 Procedure

12. Start a Wireshark capture on the active network interface.
13. Begin a large TCP download or sustained TCP stream.
14. Briefly disable and re-enable the network adapter for 1–2 seconds.
15. Repeat the same brief-disconnect test while a UDP stream is running.
16. Apply the filter `tcp.analysis.retransmission` to the TCP capture.
17. Inspect the UDP capture for recovery packets; none are generated by UDP itself.
18. Use Statistics → I/O Graph in Wireshark to visualize the throughput drop and recovery.

3.4 Sample Captured Output — TCP

No.	Time	Source → Destination	Info
145	5.201	192.168.1.10 → 93.184.216.34	[TCP Retransmission] Seq=48001 Len=1460
146	5.203	192.168.1.10 → 93.184.216.34	[TCP Retransmission] Seq=49461 Len=1460
150	5.640	93.184.216.34 → 192.168.1.10	[TCP Dup ACK] Ack=48001
162	6.812	93.184.216.34 → 192.168.1.10	ACK Ack=52381; transfer resumes normally

Sample Captured Output — TCP:

Wireshark screenshot: Capture showing [TCP Retransmission] and [TCP Dup ACK] during packet loss.

3.5 Sample Captured Output — UDP

No.	Time	Source → Destination	Info
88	5.150	192.168.1.10 → 192.168.1.20	UDP Src Port: 5004 Dst Port: 5004 Len=172
—	5.170–6.400	—	Gap in capture during outage; no retransmission entries
89	6.410	192.168.1.10 → 192.168.1.20	UDP Src Port: 5004 Dst Port: 5004 Len=172; stream continues

Sample Captured Output — UDP:

Wireshark screenshot: Capture showing the UDP packet-loss gap and continuation without retransmission.

3.6 Observations

- In TCP, unacknowledged segments are automatically resent after packet loss is detected.
- TCP Dup ACK packets indicate that the receiver is still waiting for a missing byte range.
- TCP throughput falls during the outage and recovers after the missing segments are retransmitted and acknowledged.
- In UDP, packets lost during the outage are not retransmitted, gap-filled or recovered by the transport protocol.

3.7 Why TCP Is Reliable but Slower

TCP numbers bytes, uses acknowledgments and retransmission timers, and resends data when loss is detected. This provides reliable and ordered delivery, but retransmission introduces waiting time and TCP congestion control reduces its sending rate after loss. UDP has no sequence numbers, acknowledgments or retransmission timers, so it continues with new datagrams without waiting for lost data. This reduces latency but provides no transport-layer guarantee that the lost data will be recovered.

3.8 Conclusion

The controlled packet-loss test confirms that TCP detects and recovers from packet loss using retransmission and acknowledgment mechanisms, providing complete and ordered delivery at the cost of added delay. UDP does not recover lost datagrams, trading reliability for lower latency and simpler processing.

Question 4

Use Wireshark to capture DNS traffic while resolving a domain name. Identify the query type (A, AAAA, MX) and the response received. Measure the time taken for resolution and explain why DNS typically uses UDP.

4.1 Objective

To capture DNS traffic generated while resolving a domain name, identify the resource-record query types and answers returned, measure query-to-response resolution time from Wireshark timestamps, and explain why DNS typically uses UDP.

4.2 Tools / Software Required

- Wireshark
- dig

4.3 Procedure

19. Start a Wireshark capture with the filter dns applied.
20. Flush the local DNS cache to force a fresh lookup.
21. Run separate lookups: dig example.com A, dig example.com AAAA, and dig example.com MX.
22. Stop the capture and note the Time value of each query and its matching response using the same Transaction ID.
23. Expand the DNS Queries and Answers sections to confirm the Type field and returned record.
24. Calculate resolution time as: response timestamp – query timestamp.

4.4 Sample Captured Output

No.	Time (s)	Source → Destination	Query Type	Info
1	0.000000	192.168.1.10 → 8.8.8.8	A	Standard query 0x5c11 A example.com
2	0.024118	8.8.8.8 → 192.168.1.10	A	Response 0x5c11 A example.com → 93.184.216.34
3	1.201355	192.168.1.10 → 8.8.8.8	AAAA	Standard query 0x7d2a AAAA example.com
4	1.229840	8.8.8.8 → 192.168.1.10	AAAA	Response AAAA example.com → 2606:2800:220:1:248:1893:25c8:1946
5	2.503010	192.168.1.10 → 8.8.8.8	MX	Standard query 0x9e44 MX example.com
6	2.541677	8.8.8.8 → 192.168.1.10	MX	Response MX example.com; no mail exchanger configured in sample

Sample Captured Output:

Wireshark screenshots: Captures showing DNS A, AAAA and MX queries and their responses.

4.5 Resolution Time Measurement

Query Type	Query Sent (s)	Response Received (s)	Resolution Time
A (IPv4 address)	0.000000	0.024118	24.1 ms
AAAA (IPv6 address)	1.201355	1.229840	28.5 ms
MX (mail exchanger)	2.503010	2.541677	38.7 ms

4.6 Observations

- A, AAAA and MX queries were resolved using one UDP request/response pair in the sample capture.
- The A query returned an IPv4 address, the AAAA query returned an IPv6 address, and the MX query returned the mail-exchanger record shown in the sample.
- All measured sample resolution times were below 40 ms.

4.7 Why DNS Typically Uses UDP

- Low latency: DNS questions and answers usually fit in a small datagram, so one UDP round trip is sufficient.
- High query volume: UDP avoids the memory and CPU overhead of maintaining TCP connection state for every short-lived lookup.
- DNS already has a 16-bit Transaction ID for matching responses with queries.
- If a UDP query is lost, the resolver can retry after a timeout.
- DNS can use TCP when a response is too large for the UDP payload limit or when reliable segmented delivery is required, such as zone transfers.

4.8 Conclusion

The capture shows that A, AAAA and MX records use the same lightweight DNS query/response format, with the QTYPE identifying the required record. The sample results were completed in tens of milliseconds using UDP. This demonstrates why UDP is the normal choice for fast, high-volume DNS lookups, while TCP is reserved for cases requiring larger messages or reliable segmented delivery.